Quantum Traffic Gridlock: QUBO and QAOA for two coordinated intersections

Arif, Rohit, Mehul — IBM Qiskit Fall Fest 2026, QC@IITI. Code, outputs and logs: this repository (see `README.md`). All numbers below come from files in `outputs/` and `logs/` (generated by script).

1. Model

Two intersections A and B, four 30-second slots, eight bits x = (A1..A4, B1..B4), 1 = North–South (NS) green, 0 = East–West (EW) green. Per slot and approach: add arrivals (plus, for B's linked approach, vehicles transferred from A); the green approach serves up to `capacity`; the remaining queue q is recorded; `floor(fraction·departures + 0.5)` of A's linked departures join B's linked approach next slot (transfers after slot 4 leave the model). Cost = Σ q (delay) + w_o · Σ max(0, q − storage) (overflow) + w_s · (number of phase switches at each intersection between adjacent slots). All 256 schedules are valid. Our implementation of this recursion agrees with `official/scorer.py` on all 7,936 evaluations (R and 30 random scenarios × 256 schedules; 0 mismatches, `logs/verification.json`). All reported costs come from the scorer.

2. QUBO derivation

Rule (scenario-independent). The cost is a sum of local terms, one per (intersection, approach, slot): `term = q + w_o·max(0, q − storage)`, where q is the queue after service, plus the switching cost. Causality gives each term a small *causal window*: the queue of an approach at A in slot t depends only on A's bits 1..t; for an unlinked approach at B on B's bits 1..t; for B's linked approach on B's bits 1..t and A's bits 1..t−1 (the vehicles that were transferred). The largest window has 7 bits, so each term is tabulated over at most 128 configurations of its own window; no search over full schedules is involved.

Quadratic surrogate. The `min(capacity, queue)` service and the `max(0, ·)` overflow make terms non-quadratic. Each term is replaced by its best quadratic approximation: the degree-2 truncation of its Walsh–Fourier expansion, i.e. the least-squares fit over its window with basis {1, x_i, x_i x_j}. Terms without active nonlinearities are reproduced exactly. The switching cost `w_s(x_t + x_{t−1} − 2x_t x_{t−1})` per adjacent pair is already quadratic and is added exactly. Summing the local fits gives `Q` and an offset over the **8 decision variables; there are no auxiliary variables** (`auxiliary_variables: []`), hence 8 qubits. No penalty terms are needed because every schedule is valid; the overflow weight is part of the cost itself.

Grounded and optimum-free. The construction uses only the traffic recursion and scenario parameters; the global optimum is never computed or encoded (enumeration appears only in `baselines.py` as a benchmark). **Verification:** (i) the summed local projections equal the global least-squares quadratic fit of the official cost on all 31 checked scenarios, with maximum absolute difference 1.9e-12; (ii) a test checks that `Q`, `h`, `J` in `qubo.json` give identical energies; (iii) bits outside a term's window never change it (prototype check).

Surrogate regret (official cost of the QUBO's exact minimiser versus C*): on R the minimiser is `11101010`, cost 72.0 = C*, regret 0 %; QUBO energy and official cost have correlation 0.995. Over 100 random scenarios (section-7 ranges): mean regret 2.06 %, 55 scenarios exactly 0 %, 90 % of scenarios ≤ 5 %, 98 % ≤ 25 %. The worst cases are 32.6 %, 27.0 %, 19.4 %, 17.5 % and 15.3 % (gen004, gen060, gen079, gen098, gen094): overflow nonlinearities there create higher-order interactions a quadratic surrogate cannot represent.

3. QAOA design

Mapping and Ising form. Schedule bit j sits on qubit 7 − j and is measured into classical bit 7 − j, so Qiskit's printed bitstring reads directly as A1..A4 B1..B4. With x = (1 − z)/2 the QUBO becomes `c + Σ h_i z_i + Σ_{i<j} J_ij z_i z_j` (`to_ising`). We divide energies by σ, the standard deviation under uniformly random bitstrings, which is analytic for an Ising form: $\sigma = \sqrt{(\Sigma\, h_i^2 + \Sigma\, J_{ij}^2)}$. This makes the angle scale comparable across scenarios.

Circuit. Hadamards on all qubits, then p = 4 layers of RZ(2γ_k h_i/σ) on each qubit, RZZ(2γ_k J_ij/σ) on each coupled pair, and RX(2β_k) on each qubit; the written `circuit.qasm` (R, seed 0) has 8 H, 96 RZZ (24 coupled pairs × 4 layers), 32 RZ, 32 RX and 8 measurements, depth 46 with RZZ counted as one gate (the QASM defines it as CX–RZ–CX, i.e. 192 CX).

Initialisation and the sign of β. Linear ramp γ_k = s_γ (k − ½)/p, β_k = s_β (1 − (k − ½)/p), with (s_γ, s_β) = (1.0, −0.4). The start state is the ground state of −ΣX and the mixer is RX(2β) = exp(−iβX), so an annealing-like path towards the *minimum* of the cost needs β of opposite sign to γ. **We first used (1.0, +0.4), which anneals towards the maximum: the starting normalised energy was +1.4 (random level 0), and COBYLA spent its budget recovering. This was found in review from `initial_objective` in `metrics.json` (see the AI disclosure) and fixed late on 16 Oct;** the old ramp stays in the ablation table (section 5). Scale choices followed rules fixed before running: the original (0.6, 0.6) → (1.0, +0.4) needed a mean-lift gain ≥ 0.02 on 30 random scenarios (generator seed 2026) without lowering the minimum lift (it gave 0.734 → 0.773), and the sign flip (1.0, +0.4) → (1.0, −0.4) was judged by the same rule (0.773 → 0.820; minimum 0.642 → 0.698; mean initial objective +1.39 → −0.96). Neither R nor H was used for these choices.

Training. COBYLA (scipy, `rhobeg` 0.2, `maxiter` 300) minimises the *exact* expectation of the normalised cost Hamiltonian from the statevector (no shots). Every objective call is counted and asserted ≤ 300; the runs use exactly 300 (R, seed 0: objective −0.96 → −1.40). Training is deterministic and does not depend on the seed; **the seed only sets the final sampler** (`StatevectorSampler(seed)`), 4,096 shots. Runtime is 1.28–1.39 s per seed on R (1.27–1.42 s on H) (laptop CPU, noiseless simulation).

4. Results on R and H

Scenario H was released late (about 17:40 IST on 16 Oct, Discord #doubts) and was run **unmodified** with the same command, after all settings were fixed. Values are from `outputs/{R,H}/seed*/metrics.json`; elite set = the 13 lowest-cost schedules.

seed	R E_Q	R lift	R enrichment	H E_Q	H lift	H enrichment
0	89.35	0.7810	11.67	243.02	0.8640	16.32
1	89.39	0.7806	11.64	242.85	0.8665	16.38
2	89.42	0.7802	11.76	243.28	0.8600	16.32
3	89.21	0.7829	12.02	242.83	0.8668	16.39
4	89.25	0.7824	11.92	242.90	0.8657	16.30

(R: C* = 72.0, C_rand = 151.25. H: C* = 234.0, C_rand = 300.28.) **Minimum lift: R 0.7802 (mean 0.7814); H 0.8600 (mean 0.8646).** The seed changes only the sampling noise (lift varies by under 0.01).

Section-9 baselines (official cost; schedule in brackets):

method	R	H
(1) best alternating	116.0 (`10101010`)	261.0 (`10100101`)
(2) per-slot greedy	92.5 (`01111001`)	244.0 (`10100111`)
(3) exact enumeration C*	72.0 (`11101010`)	234.0 (`10000111`)
(4) uniform-random mean	151.25	300.28
(5) exact minimiser of our QUBO (surrogate ceiling)	72.0 (`11101010`), regret 0 %	236.0 (`10010110`), regret 0.85 %
(6) raw QAOA, mean of 4,096 samples (E_Q), 5 seeds	89.21–89.42	242.83–243.28
(6) raw QAOA, lowest-QUBO-energy sample	72.0 (`11101010`), all seeds	236.0 (`10010110`), all seeds
(6) raw QAOA, most frequent sample	79.5 (`11111010`), all seeds	235.0 (`10010111`), all seeds
hybrid (1-bit-flip refinement; **not raw QAOA**)	72.0	236.0

On H the raw QAOA best sample is the QUBO's minimiser (236.0), not the true optimum (234.0): the sampler cannot beat the surrogate's ceiling. The raw mean cost beats random and the best alternating plan on both scenarios, beats the single greedy schedule on R (89.3 versus 92.5) and only narrowly on H (243 versus 244). One optimum among 4,096 shots is not evidence of performance, hence lift and enrichment.

5. Ablations

Each setting was run on 30 random scenarios (seed 2026) and R (`logs/ablations.md`); lift and enrichment from 4,096 sampled shots (seed 0). All rows below use the **adopted ramp (1.0, −0.4)** except where stated; the random-angle rows do not depend on the ramp.

setting	mean lift	min lift	mean enrichment	mean regret %	R lift
p = 1	0.563	0.421	5.29	1.77	0.620
p = 2	0.701	0.566	8.70	1.77	0.734
p = 3	0.782	0.645	11.26	1.77	0.767
p = 4 (adopted)	0.820	0.698	12.54	1.77	0.781
p = 4, random angles U(0,1), seeds 1 / 2 / 3	0.622 / 0.741 / 0.686	0.096 / 0.547 / 0.305	7.75 / 9.72 / 8.23	1.77 / 1.77 / 1.77	0.663 / 0.738 / 0.729
surrogate degree 1 (linear only)	0.418	-0.894	4.91	41.69	0.451
objective CVaR 0.25	0.769	0.536	12.43	1.77	0.774

Lift rises with depth at every step. The quadratic terms are essential: the linear-only surrogate has 41.7 % mean regret and its lift can be negative. Mean-energy training beats CVaR on mean and minimum lift; enrichment is about equal. Every random start is worse than the ramp in mean and minimum lift (minimum 0.10–0.55 versus 0.70). There are no free constraint-penalty weights (all schedules are valid; the overflow weight is a scenario parameter), so the surrogate-degree ablation is the model-side counterpart.

Before the sign fix (β with the same sign as γ, starting above the random energy level; rows kept for the record and for the selection rules in section 3):

setting	mean lift	min lift	mean enrichment	mean initial objective	R lift
p = 4, ramp (0.6, +0.6)	0.734	0.591	9.33	+1.19	0.740
p = 4, ramp (1.0, +0.4)	0.773	0.642	10.74	+1.39	0.740
p = 4, ramp (1.0, −0.4), for comparison	0.820	0.698	12.54	-0.96	0.781

Other old-sign rows (ramp 0.6, +0.6; mean lift): p = 1 / 2 / 3 = 0.563 / 0.665 / 0.719; CVaR 0.25 = 0.692; degree 1 = 0.440. The ordering of depth, objective and surrogate conclusions is the same in both blocks, but the corrected ramp is better throughout.

6. Generalisation

100 random scenarios (`scenario_gen`, seed 7, section-7 ranges, pipeline seed 0, same `run_pipeline` function as the CLI; none excluded; a different generator seed from the 30 used for selection):

metric	mean	min	10th percentile
lift	0.818	0.671	0.748
elite enrichment	12.12	5.28	7.12
QUBO regret % of C*	2.06	0.00	0.00

Before the sign fix the same study gave mean lift 0.782 and minimum 0.607 (git history, commit 07d9000). Lift is measured against the true C*, so it stays high even where the surrogate is imperfect (gen004, regret 32.6 %, lift 0.881).

Our own held-out scenarios (random, `scenario_gen` seed 99, in `scenarios/`; **our test scenarios, not the official H**), seeds 0–4:

scenario	C*	C_rand	QUBO regret	min lift	enrichment range	greedy	best alternating
heldout_1	188.0	227.36	0 %	0.8296	12.99–13.05	199.0	203.0
heldout_2	277.5	318.22	0 %	0.8035	5.74–6.00	281.0	290.5

The sign fix is not uniformly better: heldout_2 got worse (min lift 0.844 → 0.804, enrichment 8.2–8.4 → 5.7–6.0), while heldout_1 improved (0.808 → 0.830).

7. Limitations

- We shipped the wrong-sign ramp until a late review of `initial_objective`; a regression test now requires the starting energy to be below the random level. The ablations were rerun on the corrected ramp; the old-sign rows are kept separately.
- COBYLA uses the whole 300-evaluation budget on every p = 4 run, so the angles are not converged.
- The quadratic surrogate fails on some scenarios: regret up to 32.6 % (10 of 100 scenarios exceed 5 %); on H it costs 2.0 units (0.85 %) and caps the raw best sample at 236.0. Higher-order terms or auxiliary variables were not built.
- The ramp choices were made sequentially on the same 30 scenarios, so part of the gain is likely selection noise; the independent 100-scenario set (0.782 → 0.818) and R (0.739 → 0.781 mean lift) point the same way, but heldout_2 worsened.
- All results are noiseless statevector simulations; no noise model or hardware. Ablations used one sampler seed; mean-lift differences below about 0.02 should not be over-interpreted.
- The hybrid result is reported separately and is not part of the raw counts. We make no quantum-advantage claim: there are only 256 candidate schedules.

8. Reproduce

```
pip install -r requirements.txt          # Python 3.13.9, qiskit 2.5.2, numpy 2.5.4, scipy 1.18.1
    tested
python run.py --scenario official/scenario_R.json --seed 0 --out outputs/R/seed0   # H:
    official/scenario_H.json -> outputs/H/seed0
bash make_outputs.sh                     # outputs/R and outputs/H (seeds 0-4)
python -m unittest discover -s tests     # 20 tests
python ablations.py; python generalisation.py; python report_numbers.py
python official/scorer.py --scenario official/scenario_R.json --schedule outputs/R/seed0/schedule.json
```